

UNIVERSITY OF CLAUDE · FREE RESOURCE MAP

Resource Map

CSCE 614 Computer Architecture — module-by-module mapping onto free university courseware

HOW TO USE THIS

This is your lecture hall. For each module: watch or read the mapped external material first, then work the module slides and notes, which are written to assume you have seen it.

Where a specific lecture number is given, that is the one that covers this module. Where a book chapter is given, read that chapter, not the whole book.

Everything here is free. Resources remain under their own licences — link to them, do not redistribute them.

Primary sources for the whole course

Resource	Role in this course
ETH Zürich — Onur Mutlu, Computer Architecture (full lecture videos, free)	Primary lecture series. Graduate level, exceptionally thorough, and freely available in full with slides and assignments.
MIT 6.004 Computation Structures (OCW, Spring 2017)	The foundation course: from transistors to a working pipelined processor. Start here if digital logic is unfamiliar.
MITx 6.004.1x Computation Structures: Digital Circuits (edX)	The same material in graded MOOC form, free to audit.
Nand2Tetris — build a computer from NAND gates (free)	If you have never built a processor, do the first six projects. The understanding it produces is disproportionate to the effort.
Ulrich Drepper — What Every Programmer Should Know About Memory (free)	The definitive practical treatment of the memory hierarchy. Long, dense, and worth every page. Core reading for Modules 05–07.
Agner Fog — Software optimization manuals (free)	Microarchitecture tables, instruction latencies, and optimisation guidance for every recent x86 core. The reference you will keep.

Module-by-module map

Module 01 — Performance: What to Measure

Resource	What it covers for this module
Onur Mutlu — Computer Architecture, Lecture 1–2 (Introduction, Performance)	The framing of the course and the performance equation, done thoroughly.
MIT 6.004 — Performance Measures	The equation and Amdahl's law at an accessible level.
Latency Numbers Every Programmer Should Know	The table from §5 with more entries. Memorise the orders of magnitude.
Brendan Gregg — perf Examples	The practical reference for Linux performance counters. Bookmark it.
Agner Fog — Optimizing software in C++	Chapter 2 covers benchmarking methodology properly.

Module 02 — Instruction Sets and the Hardware/Software Interface

Resource	What it covers for this module
Onur Mutlu — ISA, RISC vs CISC lectures	The design-space discussion done properly, including why the historical debate resolved as it did.
MIT 6.004 — Instruction Sets, Assembly Language	Builds an ISA from scratch and then a processor to run it. The best way to understand what an ISA is for.
Compiler Explorer	Source and assembly side by side for any compiler and flags. Use it in every remaining module.
Agner Fog — Instruction tables	Latency and throughput for every instruction on every recent x86 microarchitecture. The reference when you need an exact number.
RISC-V specifications (free)	A modern ISA designed in the open, and short enough to read. Instructive by contrast with x86.

Module 03 — Pipelining and Hazards

Resource	What it covers for this module
Onur Mutlu — Pipelining, Pipeline Hazards lectures	The full treatment including forwarding paths and the depth trade-off.
MIT 6.004 — Pipelined Processors	Builds a pipelined processor stage by stage. The best way to make forwarding concrete.
Nand2Tetris Projects 1–6	Build a CPU from gates. Not pipelined, but the understanding transfers.
Agner Fog — The microarchitecture of Intel, AMD and VIA CPUs	What real pipelines look like, core by core. Dense, and the definitive free reference.

Module 04 — Branch Prediction and Speculation

Resource	What it covers for this module
Onur Mutlu — Branch Prediction lectures	Static through TAGE, with the correlation insight developed properly.
Dan Luu — Branch prediction	A short, clear history of predictor designs with the intuition for each.
Stack Overflow — 'Why is processing a sorted array faster?'	The canonical demonstration, with excellent answers. Run it yourself before reading.
Spectre Attacks: Exploiting Speculative Execution (free paper)	The original paper. Readable, and worth reading in full for how carefully the abstraction is picked apart.

Module 05 — Caches: The Memory Wall

Resource	What it covers for this module
Ulrich Drepper — What Every Programmer Should Know About Memory	Sections 2–3 cover this module in far greater depth. The single most valuable free document on the subject.
Onur Mutlu — Memory Hierarchy and Caches lectures	Organisation, the three Cs, and replacement policies.
MIT 6.004 — The Memory Hierarchy	A clean introduction to associativity and address breakdown.
Igor Ostrovsky — Gallery of Processor Cache Effects	Nine short experiments, each isolating one cache behaviour. Run all of them — it takes an afternoon and is worth a week of reading.

Module 06 — Cache Optimization and Prefetching

Resource	What it covers for this module
Ulrich Drepper — What Every Programmer Should Know About Memory	Section 6 is a catalogue of exactly these optimisations with measured results. The core reading for this module.
Onur Mutlu — Prefetching lectures	Hardware prefetcher designs and their limitations.
Igor Ostrovsky — Gallery of Processor Cache Effects	Nine experiments isolating individual effects. Run every one.
Mike Acton — Data-Oriented Design and C++ (CppCon, free video)	The practitioner's argument for layout-first thinking, from a game engine perspective. Directly relevant to CSCE 641.

Module 07 — Virtual Memory and the TLB

Resource	What it covers for this module
Onur Mutlu — Virtual Memory lectures	Translation, TLBs, and the interaction with caches.
Ulrich Drepper — What Every Programmer Should Know About Memory, section 4	Virtual memory and TLB behaviour with measurements.

Resource	What it covers for this module
OSTEP — Virtual Memory chapters (free book)	The operating-system side: paging policy, replacement, swapping. Overlaps usefully with CSCE 611.
Linux kernel documentation — Transparent Hugepage Support	How THP works in practice, and why some workloads disable it.

Module 08 — Out-of-Order Execution

Resource	What it covers for this module
Onur Mutlu — Out-of-Order Execution, Tomasulo's Algorithm	Renaming, reservation stations, and the reorder buffer developed in full.
Agner Fog — The microarchitecture of Intel, AMD and VIA CPUs	Actual window sizes, physical register counts, and execution port layouts for every recent core.
Travis Downs — performance blog	Careful microbenchmark-driven investigations of out-of-order behaviour. An excellent model of how to measure this material.
Intel — Top-Down Microarchitecture Analysis Method	A systematic method for attributing stalls to front end, back end, bad speculation, or retiring. The professional version of \$5.

Module 09 — SIMD and Data-Level Parallelism

Resource	What it covers for this module
Agner Fog — Optimizing subroutines in assembly language	The reference for vector instruction selection, latencies, and throughputs.
Intel Intrinsics Guide	Searchable reference for every intrinsic, with latency and throughput per microarchitecture.
Onur Mutlu — SIMD Processing lectures	Vector processors and SIMD from the architectural side, including the historical vector supercomputers.
Mike Acton — Data-Oriented Design and C++ (free video)	Why layout decisions dominate, argued from shipped game engine experience.

Module 10 — GPUs as Throughput Machines

Resource	What it covers for this module
NVIDIA CUDA C++ Programming Guide	The authoritative reference. Chapters on the execution model and memory coalescing are the ones that matter here.
OLCF CUDA Training Series (free video + exercises)	A structured free course with hands-on labs. The best guided introduction available.
Onur Mutlu — GPUs and SIMT lectures	The architectural view, with the latency-hiding argument developed properly.

Resource	What it covers for this module
CMU 15-418 / Stanford CS149 — GPU architecture lectures	Kayvon Fatahalian's treatment, which is unusually good at explaining why GPUs are shaped the way they are. Also the basis of CSCE 735.

Module 11 — Multicore and Cache Coherence

Resource	What it covers for this module
Onur Mutlu — Cache Coherence, Multiprocessors lectures	MESI and directory protocols developed properly, with the scaling arguments.
Ulrich Drepper — What Every Programmer Should Know About Memory, section 3.3	Coherence and false sharing with measurements.
Paul McKenney — Is Parallel Programming Hard? (free book)	The practitioner's treatment of sharing, counters, and scalability. Chapter 5 on counting is directly relevant and unusually good.
perf c2c documentation	The tool that finds false sharing by cache line and source line.

Module 12 — Memory Consistency and Synchronization

Resource	What it covers for this module
Onur Mutlu — Memory Consistency lectures	The hardware side: store buffers, consistency models, and fences.
Paul McKenney — Is Parallel Programming Hard? (free book)	The definitive free treatment of memory ordering in practice, from the author of the Linux RCU implementation.
Jeff Preshing — blog on lock-free programming	The clearest available explanations of acquire/release semantics, with worked examples and diagrams. Start with 'Memory Ordering at Compile Time'.
Russ Cox — Hardware and Programming Language Memory Models	A three-part series covering both hardware and language models, unusually clearly.
cppreference — <code>std::memory_order</code>	The precise semantics, with examples. The reference to check before writing any atomic code.

Module 13 — Energy, Scaling, and Specialization

Resource	What it covers for this module
Onur Mutlu — Why Computer Architecture? and the closing lectures	The scaling story and the case for specialisation, from someone who works on it.
Hennessy & Patterson — 'A New Golden Age for Computer Architecture' (Turing lecture, free)	The authors of the standard textbook on where architecture goes after Dennard scaling. Short, and the best single summary of this module's argument.

Resource	What it covers for this module
Herb Sutter — The Free Lunch Is Over (2005)	The essay that told the software industry the transition had happened. Worth reading as a historical document.
Mark Horowitz — Computing's Energy Problem (ISSCC keynote, free slides)	Where the energy actually goes in a modern core. The source of the numbers in §3.